Visualization of Sound Spectra of a Whistle

Jorge R. Gherson

February 28, 2026

Contents

1 General Objective **3**

2 Exploratory Analysis of the Sound Spectra Dataset **3**

3 Mathematical Formulation **5**

1 General Objective

We have a dataset containing the sound spectra of a whistle collected at different mass flow rates, pressures, and frequencies. The main objective of this project is to visualize the sound spectra in a way that would allow a Convolution Neural Network (CNN) or a human to efficiently see the relationship between the sound spectra and the mass flow rate.

2 Exploratory Analysis of the Sound Spectra Dataset

The initial data analysis was carried out in a Jupyter notebook named `initial_analysis.ipynb`. First, it is necessary to import the required libraries and load the dataset. The dataset was given in a `.mat` file, which was then loaded using the `scipy.io` library. We have a dataset that contains 267 measurement instances for the following variables:

1. P_{all}: A 2D array containing the sound pressure (in Pascals) for each frequency and time point.
2. `amplitude_all`: A 2D array containing the amplitude of the sound for each frequency and time point.
3. `frequency_all`: A 2D array containing the Fast Fourier Transform (FFT) frequencies with 49902 points.
4. `m_dot_all`: A 2D array containing the mass flow rate (in kg/s) for each frequency and time point.
5. `signal_all`: A 2D array containing the 1 second worth of the original microphone signal collected at 100,0000 Hz.

First, I sought to understand the structure of the dataset by checking the shapes of each variable. The shapes of the variables are as follows:

```
P_all (267, 1)
amplitude_all (267, 1)
frequency (49902, 1)
m_dot_all (267, 1)
signal_all (267, 1)
```

Notice all of the lengths agree, except for the frequency variable, which has a different shape. This is because the frequency variable contains the FFT frequencies, which are the same for all samples, while other variables contain data for each sample. Then, I evaluated if the 267 amplitudes correspond with every Nth frequency, where N is the number of frequencies. Finding the ratio between data points in the frequency variable and the amplitude variable gave us a ratio of 186. This means that the amplitude variable is not evenly distributed across the frequencies, which is an important observation for the visualization process.

Then, I proceeded to unpack each variable and asign it to a respective one in Python to evaluate how the data was structured, finding that data stored in the `.mat` file is passed as a dictionary in Python (Note: not sure.). Running for `amplitude_all` yielded:

```
array([[array([[8.72111064e-05],
            [3.07487090e-04],
            [2.17788468e-04],
            ...,
            [4.34430331e-06],
            [4.19685731e-06],
            [1.59920000e-06]], shape=(49902, 1))],
    [array([[3.06550360e-04],
            [6.16053321e-04],
            [2.33531321e-04],
            ...,
            [3.32547277e-06],
            [1.64016323e-06],
            [9.15600000e-07]], shape=(49902, 1))],
```

```
        [array([[1.70402252e-04],
                [1.92546836e-04],
                [2.33929164e-04],
                ...,
                [2.84986090e-06],
                [2.47268173e-06],
                [1.11950000e-06]], shape=(49902, 1))],
        [array([[3.62037875e-04],
                [3.37830354e-04],
                [1.09858487e-03],
                ...,
...
                [9.88551412e-04],
                ...,
                [3.28158474e-05],
                [3.24278557e-05],
                [3.83440000e-06]], shape=(49902, 1))]], dtype=object)
```

Running so for `signal_all` yielded similar results. Notice that both objects have a shape (49902, 1), so it makes sense to make the spectrum 1D and adding them vertically. For the amplitudes:

```
amplitude_all = np.vstack([
        amplitude_all[i].flatten()
        for i in range(len(amplitude_all))
])
```

And for the frequecies:

```
signal_all = np.vstack([
        sig.flatten()
        for sig in signal_all
])
```

Which yielded:

```
Amplitude Shape: (267, 49902)
Amplitude Dimension: 2
Amplitude dtype: float64
```

and

```
Signal Shape: (267, 100000)
Signal Dimension: 2
Signal Data Type: float64
```

Now it is necessary to note that, for frequency analysis it is convenient to implement a logarithmic scale, whhich was calculated using:

$$\log(\nu) = 20 \log\left(A + 1 \times 10^{-12}\right) \tag{1}$$

where ν represents the frequency and A the amplitude. We also add 1×10^{-12} to take noise into account. These were then normalized using

$$\nu_{\text{norm}} = \log \nu - \max \log \nu. \tag{2}$$

Then, peak frequencies were extracted by implementing masking to crop the data. In order to visualize the relationsip, we must sort the mass flow rates and frequencies. Then, I defined a sampling rate of 48000 Hz, which I used as a parameter for the Fast Fourier Transform (FFT) applied using Welch's method. Then, I converted the power to decibels (dB) using a logarithmic scale, and I normalized them relative to the maximum. These were afterwards sorted, and plotted using a heatmap that yielded:

Notice for this graph that, if the oscillations

Figure 1: The spectrum plotted against the mass flow rate in a logarithmic scale after having applying Welch's method for FFTs.

3 Mathematical Formulation

In order to make the relationship easily visible for a Convolution Neural Network, we will carry out a mathematical definition for how the network transforms the raw acoustic spectrum into a mass flow rate prediction.

Let our normalized acoustic spectrum for a single flow rate be defined as a one-dimensional vector

$$\mathbf{x} = \begin{bmatrix} x_1 & x_2 & \cdots & x_N \end{bmatrix}^T \tag{3}$$

where N is the total number of frequency bins. The core operation of a CNN is the discrete 1D convolution, which applies a set of K trainable kernels (i.e., filters) to the input to detect features like whistle peaks or harmonic intervals.

Let a single filter be a vector $\mathbf{w}^{(k)}$ of length M. The convolution operation computes a new sequence $\mathbf{z}^{(k)}$ for this filter. The i-th position is given by:

$$z_i^{(k)} = \sum_{j=1}^{M} \mathbf{x}_{i+j-1} \cdot \mathbf{w}_j^{(k)} + b^{(k)} \tag{4}$$

where $\mathbf{w}_j^{(k)}$ represents the weights of the k-th filter, and $b^{(k)}$ the bias of the k-th filter. This represents a mathematical cross-correlation. When the shape of the filter matches the shape of a peak in the spectrum, z_i spikes. To allow the network to learn complex, non-linear physical relationships, we apply a Rectified Linear Unit (ReLU) to the output of the convolution:

$$h_i^{(k)} = \max\left(0, z_i^{(k)}\right). \tag{5}$$

Then, pooling reduces the size of the data, making the model computationally efficient and "translation invariant." The maximum pooling takes the maximum value over a sliding window of size P:

$$p_i^{(k)} = \max h_{P(j-1)+1}^{(k)}, \cdots, h_{Pj}^{(k)}. \tag{6}$$

Later in the network, we use global average pooling (GAP), which takes the average of the entire remaining sequence of length L for each filter k, resulting in a single scalar value per filter:

$$g^{(k)} = \frac{1}{L} \sum_{i=1}^{L} h_i^{(k)}, \tag{7}$$

which flattens the matrix into a vector $\mathbf{g} = \begin{bmatrix} g^{(1)} & g^{(2)} & \cdots & g^{(K)} \end{bmatrix}^T$. Once the spatial features (i.e., peaks, harmonics) are extracted and flattened into the vector $\mathbf{g}$, a standard neural network layer combines them to predict the flow rate. Let $\mathbf{W}_d$ be the weight matrix of the dense layer and b_d be the bias vector of the dense layer. Then,

$$\mathbf{d} = \max\left(0, W_{d\mathbf{g}} + b_d\right). \tag{8}$$

Since we are predicting a continuous physical variable, the final layer is a single node with a linear activation function:

$$\hat{y} = \mathbf{w}_{\text{out}}^T d + b_{\text{out}} \tag{9}$$

where $\hat{y}$ represents the predicted mass flow rate y and $\mathbf{w}_{\text{out}}$ represents the final weight vector mapping the learned features to the prediction. However, in order to train a network efficiently, we must also define how "wrong" our predictions can be compared to the actual mass flow rate. For regressions, we usually use Mean Squared Error (MSE) over a batch of B training samples:

$$L = \frac{1}{B} \sum_{i=1}^{B} (\hat{y}_i - y_i)^2 \tag{10}$$